BAIL security

Dowsure

# FINAL REPORT

May ‘2026

# Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

# 1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

| Project | Dowsure - Audit Report |
| --- | --- |
| Website | dowprotocol.com |
| Language | Solidity |
| Methods | Manual Analysis |
| Github repository | https://github.com/vision77/dow-contracts-new/tree/50eae73eff46cda5b0fa6d5fd060b6e454d8cc43/contracts/version2 |
| Resolution 1 | https://github.com/dowprotocol/dow-contracts/tree/db9ceb477f181e9927128ae04f9dcacfee0324e2 |
| Resolution 2 | https://github.com/dowprotocol/dow-contracts/commit/4de794415f128d694373c1bddfca9c5c5f85e650 |

## 2. Detection Overview

| Severity | Found | Resolved | Partially Resolved | Acknowledged (no changes made) | Failed resolution | Open |
|---|---|---|---|---|---|---|
| High | 4 | 4 | | | | |
| Medium | 11 | 6 | 2 | 3 | | |
| Low | 11 | 6 | | 5 | | |
| Informational | 14 | 5 | | 9 | | |
| Governance | 5 | 2 | 1 | 2 | | |
| Total | 45 | 23 | 3 | 19 | | |

## 2.1 Detection Definitions

| Severity | Description |
|---|---|
| High | The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users. |
| Medium | While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences. |
| Low | Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately |
| Informational | Effects are small and do not post an immediate danger to the project or users |
| Governance | Governance privileges which can directly result in a loss of funds or other potential undesired behavior |

# 3. Detection

## VaultConfig

The VaultConfig contract is the configuration and permission layer for a vault. It manages the settings that the rest of the system reads from VaultConfigStore, including supported underlying assets, asset decimals, max capacity, base APY, the first epoch start time, and the early exit settings used during redemption flows. After deployment, the vault owner can update most configuration values, while other contracts use the helper checks here to confirm that the protocol is not paused and that the linked vault is still open before allowing state-changing operations.

### Privileged Functions

- initialize
- initializeWithUnderlyingAssets
- setVault
- setVaultName
- setNormalizedDecimals
- setUnderlyingAssetSupported
- setBaseApy
- setMaxCapacity
- setFirstEpochStartAt
- setEarlyExitImmediatePrincipalBps
- setEarlyExitImmendiateClaimDelay
- setEarlyExitYieldPenaltyBand

### Core Invariants:

INV 1: Initialization can only succeed once, and it must set a nonzero DowConfig, at least one supported underlying asset, a nonzero max capacity, and a first epoch start time that is still in the future.

INV 2: Every supported underlying asset must also be allowed by DowConfig, and its token decimals cannot exceed the configured normalized decimals.

INV 3: Strategy manager, emergency redeem requester, and emergency redeem reviewer permissions are determined by the roles held in this contract and are the source of truth for downstream access checks.

INV 4: Early exit settings must remain within valid basis point bounds, and the penalty schedule must stay inside the six defined staking bands.

INV 5: The protocol and vault health check should only pass when the protocol is not paused, a vault address has been set, and the linked vault reports that it is open.

| Issue_01 | Accounting breaks if normalizedDecimals is updated in VaultConfig |
|---|---|
| **Severity** | **Governance** |
| **Description** | The VaultConfig has a setter function that allows the vault owner to update the normalized decimals. However, if this value is updated, the existing normalized deposit amount stored in totalValueLocked becomes incorrect. This would lead to accounting errors on subsequent deposits and withdrawals. If normalizedDecimals is increased, this would lead to a revert on withdrawals.<br><br>*function setNormalizedDecimals(uint256 _normalizedDecimals) external virtual {*<br>*onlyVaultOwner(msg.sender);*<br>*if (_normalizedDecimals == 0) revert Errors.InvalidZeroAmount();*<br>*store.setNormalizedDecimals(_normalizedDecimals);*<br>*}*<br><br>Other risks also exist when normalized decimals are updated:<br>- Function _normalizeAmount in VaultCommonBase assumes normalizedDecimals is greater than or equal to the underlying asset decimals. While this is verified in the VaultConfig when adding an underlying asset, the normalizedDecimals value can be changed after this (reduced), which can cause an underflow.<br>- Function _assetToDrtAmount can underflow if decimals are greater than DRT_DECIMALS (18). For example, normalizedDecimals is updated to 24 or 36. This allows the addition of underlying assets with decimals greater than 18, which eventually underflow in _assetToDrtAmount, which bricks deposits and withdrawals. |
| **Recommendations** | Consider acknowledging this issue or removing the setter function that allows the owner to alter normalized decimals. |
| **Comments / Resolution** | Fixed by removing the setNormalizedDecimals function. |

| Issue_02 | Disabling a live asset can lead to stuck funds |
|---|---|
| **Severity** | **Governance** |
| **Description** | Disabling an underlying asset clears support and decimals metadata even if positions in that asset are still active due to the logic in _setUnderlyingAssetSupported. Live positions still rely on that data for DRT burn math, TVL normalization, and liquidity transfers from Vault into RedeemVault. If the asset is disabled while exposure remains, claim and accounting paths can start reverting or become impossible to pay. |
| **Recommendations** | Consider documenting and acknowledging the impacts of this issue. |
| **Comments / Resolution** | Acknowledged. |

| Issue_03 | setUnderlyingAssetSupported cannot be used to unset assets if already disallowed |
|---|---|
| **Severity** | **Low** |
| **Description** | The setUnderlyingAssetSupported function has the following check.<br><br>*if (!dc.isAllowedLiquidityAsset(_underlyingAssets[i])) {*<br>*revert Errors.InvalidUnderlyingAsset();*<br>*}*<br><br>This check runs irrespective of whether enabled is being passed in as true or false. Thus, if an asset is marked disallowed by the dowconfig, the setUnderlyingAssetSupported function cannot be used to mark it as disallowed in the vaultconfig due to this check. |
| **Recommendations** | Consider skipping this check if enabled is false, i.e., if the asset is being disallowed. |
| **Comments / Resolution** | Fixed by skipping the dc check if the asset is being disabled. |

## EpochManager

The EpochManager contract manages the lifecycle of protocol epochs while using EpochManagerStore as the source of stored epoch data. It splits authority between an admin and authorized vaults. The admin can change the admin role and decide which vaults are authorized, while authorized vaults are responsible for creating epochs, opening or closing them, selecting the active subscription epoch, and settling final epoch results.

When creating an epoch, the contract enforces a future start time, non-zero duration values, a valid subscription and claim window, and an APY that does not exceed the basis point denominator. It also prevents duplicate start timestamps and disallows a new epoch from overlapping the latest existing epoch. After creation, vaults can mark an epoch as open, set it as the current subscription epoch only if it is open and has not started yet, and settle it once by recording the final APY and bad debt. The contract also includes helper calculations for effective staking duration, accrued yield, and forfeited yield.

### Privileged Functions

- setAdmin
- authorizeVault
- createEpoch
- setEpochOpen
- setSubscriptionEpoch
- settleEpoch

### Core Invariants:

INV 1: Only the admin can update the admin address or change vault authorization, and only authorized vaults can create, manage, or settle epochs.

INV 2: Each epoch start time must be unique, and newly created epochs cannot begin before the latest existing epoch has ended.

INV 3: The subscription epoch must point to an existing opened epoch whose start time is still in the future, and it cannot replace another active subscription epoch before that epoch has ended.

INV 4: An epoch can only be settled once, and the recorded APY and bad debt values must remain within the basis point denominator.

| Issue_04 | All redemption paths are blocked until settlement |
|---|---|
| **Severity** | **Governance** |
| **Description** | The settleEpoch function needs to be called to update the state of the epoch. However, if this function is not called, all redemption paths stay blocked. earlyWithdraw cannot be called, since the eventual _principalAfterEpochBadDebt fails if settlement is not complete. The same is true for the fulfillClaim functions. Thus, all redemptions require settlement calls first. |
| **Recommendations** | Consider acknowledging this issue. |
| **Comments / Resolution** | Acknowledged. |

| Issue_05 | No timelocks on settleEpoch |
|---|---|
| **Severity** | **Governance** |
| **Description** | The settleEpoch function does not check for any timestamps. Any epoch can be settled at any time, in the past or in the future. This means that the strategy manager can settle an epoch that hasn't even started yet, and set its bad debt percentage to any value. |
| **Recommendations** | Consider making sure only epochs in the past can be settled, or acknowledging this issue |
| **Comments / Resolution** | Fixed by only allowing epochs to be settled after the end. |

| Issue_06 | Missing events when changing states |
|---|---|
| **Severity** | **Informational** |
| **Description** | It is common practice to emit an event when a transaction changes a storage state. In multiple contracts throughout the protocol, state changes occur, but no events are emitted. As an example, the setAdmin function changes the value of the admin storage variable, but no event is emitted. This same issue persists in multiple functions and contracts throughout the system. |
| **Recommendations** | Consider emitting events whenever a variable is changed. |
| **Comments / Resolution** | Acknowledged. |

| Issue_07 | Unused claimEndAt functionality |
|---|---|
| **Severity** | **Informational** |
| **Description** | The EpochManager contract defines a claimEndAt value when creating an epoch. However, this value is never used. Redemptions only check to make sure the epoch end time has passed; it doesn't check for any claim expiry timestamp. |
| **Recommendations** | Either implement the functionality or remove the claimEndAt values. |
| **Comments / Resolution** | Acknowledged. |

| Issue_08 | Missing documentation for affecting staking duration |
|---|---|
| **Severity** | **Informational** |
| **Description** | Currently, it is unclear how the effective staking duration is intended to be calculated.<br><br>In _calcEarlyWithdrawTerms from RedeemVault, calcAt = block.timestamp and epochEndAt = interestEndAt. Now in this case, if block.timestamp is less than interestEndAt, stakeDuration = block.timestamp - epochStartAt and epochDuration = interestEndAt - epochStartAt, hence remaining equals to interestEndAt - block.timestamp. Now, if the stakeDuration (time since start) is less than the remaining time pending till interestEndAt, the staking duration is 0 and no yield is accrued (this means we need to be atleast 50% through the epoch to earn yield). If it is greater than the remaining time (more than 50% but less than 100%), this would subtract that remaining time from stakeDuration. So if we're 80% (stakeDuration) from epoch start to interestEndAt, then 20% (remaining) is subtracted from the stakeDuration, making the effective time 60%. This affects yield calculation.<br><br>In this case:<br>- If a user attempts to withdraw before half the epoch is complete, zero yield is accrued.<br>- If a user attempts to withdraw before the full epoch is complete, the user receives yield based on a reduced staking duration. |
| **Recommendations** | Consider documenting this behaviour if intended. |
| **Comments / Resolution** | Acknowledged. |

## Vault

The Vault contract is the main user-facing entry point for deposits, position tracking, and epoch-based principal accounting. Users deposit supported assets during the active subscription window of the current subscription epoch, receive epoch-specific DRT balances as receipt tokens, and get a position recorded in VaultStore with principal, shares, lockup timing, and auto roll preference.

The contract also handles most operator-side controls for the product. Strategy managers can create and manage epochs, refresh APY, move funds in and out for lending, and batch roll positions that opted into auto roll.

Vault also serves as the funding source for RedeemVault when redemptions need liquidity. For each epoch, it deploys a separate DRT token, keeps total TVL and per asset TVL in sync with deposits and bad debt adjustments, and moves DRT balances across epochs when positions roll forward.

### Privileged Functions

- setDrtRedeemVault
- setActivityLogger
- setEpochManager
- configureEpoch
- setEpochOpen
- setSubscriptionEpoch
- settleEpoch
- refreshApy
- strategyDeposit
- withdrawForLending
- rollPositions
- setVaultStatus
- emergencyWithdrawUnderlying
- reviewEmergencyWithdrawUnderlying
- fundRedeemVault

### Core Invariants:

INV 1: New deposits can only enter the single active subscription epoch, and that epoch must be open and inside its subscription window.

INV 2: Every deposit mints DRT shares for the selected epoch and records a matching position with the same owner, asset, principal, shares, and lockup period.

INV 3: Auto roll only moves active opted-in positions into an open epoch whose start time matches the current epoch end time.

INV 4: Total vault TVL and per asset TVL must stay consistent with deposits, lending withdrawals, strategy repayments, redemptions funded through RedeemVault, and bad debt adjustments.

INV 5: Only authorized roles can configure epochs, move lending liquidity, execute emergency withdrawals, or pull redemption liquidity from the vault.

| Issue_09 | Emergency requester and reviewer can collude to drain the protocol |
|---|---|
| **Severity** | **Governance** |
| **Description** | The emergencyRedeemRequester role can request emergency redemptions, and the emergencyRedeemReviewer role can approve them. These 2 roles together can empty out the entire vault. The contract does not even force the protocol to be paused at this state, and can thus be drained at any point of time. |
| **Recommendations** | Consider forcing the protocol to be paused first for emergency withdrawals, and ensure the requester and redeemer role actions are behind timelocks. |
| **Comments / Resolution** | Partially fixed. Emergency withdrawals are only active after vault is closed permanently. |

| Issue_10 | Emergency withdrawals don't update the state |
|---|---|
| **Severity** | **Medium** |
| **Description** | The emergencyWithdrawUnderlying and reviewEmergencyWithdrawUnderlying functions can be used to withdraw funds from the vault in case of an emergency. However, these functions do not change the state of the position, i.e., they don't burn vault shares or change the position principal to 0.<br><br>Thus, users who exit the system via an emergency withdrawal can also then exit the system via a normal withdrawal, since their state is still left untouched. |
| **Recommendations** | Only allow emergency withdrawals after the protocol is paused. Do not unpause again, since the accounting is broken after emergency operations. Otherwise, consider carrying out the necessary state changes in the emergency functions to prevent this scenario. |
| **Comments / Resolution** | Partially fixed. Emergency withdrawals are only active after vault is closed permanently. |

| Issue_11 | Vault state can be incorrect in case of token depegging |
|---|---|
| **Severity** | **Medium** |
| **Description** | The vault contract supports multiple underlying assets and mints out the same share token. Furthermore, the TVL is updated based on the deposit amount of each token.<br><br>*uint256 received = ua.balanceOf(address(this)) - cashBefore;*<br>*uint256 normalizedReceived = _normalizeAmount(ua, received);*<br>*store.setTotalValueLocked(store.totalValueLocked() + normalizedReceived);*<br><br>This implies that all the underlying tokens are valued at the same price. However, in case one of the underlying tokens depeg, then these stored metrics lose any meaning. Also, if the vault accepts differently valued assets, like USDC and WETH, then these metrics are also meaningless. The same is true for the maxCapacity value.<br><br>Furthermore, in case of a depeg or different valued tokens, the badDebtBps setting can also cause issues. This is because the vault as a whole has a single haircut rate, instead of a per-token rate. Thus, all tokens, irrespective of price, get decreased by the same amount, affecting higher value tokens more than lower value tokens. |
| **Recommendations** | Ensure same/similarly priced tokens are used in the vault. In case of a depeg, this can still lead to large drifts. |
| **Comments / Resolution** | Acknowledged. Similarly priced stablecoins will be used. |

| Issue_12 | Yield payments are not enforced |
|---|---|
| **Severity** | **Medium** |
| **Description** | The strategy manager is expected to borrow funds via withdrawForLending and repay via strategyDeposit, and these functions keep track of the principal amount loaned out. However, there is no code in any of the contracts to ensure they are actually paying back the yields as well.<br><br>In fact, strategy managers cannot use the strategyDeposit function to put yield funds into the protocol, since there's a borrow amount check.<br><br>*if (normalizedRepaid > borrowedAmount) {*<br>*revert Errors.InvalidRepayAmount();*<br>*}*<br><br>So if yield is added through this function, this check will trigger and revert.<br><br>If no yield is added, the contract remains unaware of this situation and still lets users claim yield via the claimYield function. In such a scenario, normal protocol funds are given out to claimers, which can lead to insolvency in the protocol. |
| **Recommendations** | Consider enforcing regular yield payments in the contract by the strategy managers, or acknowledge this issue to be the responsibility of the strategy manager. |
| **Comments / Resolution** | Acknowledged. Strategy manager is trusted. |

| Issue_13 | Short periods of downtime between epochs |
|---|---|
| **Severity** | **Low** |
| **Description** | After each epoch, the subscriptionEpochId needs to be updated so that deposits can start again for the next epoch. This is done via the setSubscriptionEpoch function. However, this is only callable after the current epoch ends, due to the following check.<br><br>*if (block.timestamp < currentEndAt) {*<br>*revert Errors.InvalidEpochProgression();*<br>*}*<br><br>So this can only be called after the current epoch ends, and deposits cannot start until this is called. Thus, after every epoch, there will be a short period of time where deposits cannot take place, and the admin has to send a transaction to set this and start it again.<br><br>A similar period of inactivity is observed in the redeemVault, where, after an epoch ends but before the settleEpoch is called for the epoch that just ended, all redemptions are stopped. This is because all redemption flows need the bad debt APR of the past epoch, which needs to be set via the settle call. |
| **Recommendations** | Consider acknowledging and documenting this issue, or create a way to queue the subscription epoch ID so that once the current epoch ends, the queued epoch ID will be used for newer deposits. |
| **Comments / Resolution** | Acknowledged. |

| | |
|---|---|
| Issue_14 | Position might not generate yield if not rolled over by the strategy manager |
| **Severity** | **Low** |
| **Description** | The strategy manager role is expected to call the rollPositions function to roll over positions to newer epochs. However, if they miss a position by mistake or maliciously, that position can stop earning yield.<br><br>This is because neglected positions can enter a situation where, once they get rolled, the new position is also matured. In such a scenario, the autoroll value is set to false, and the position stops getting rolled or earning any yield.<br><br>Thus, positions neglected for multiple epochs won't get any yield, while the strategy manager still retains access to their funds, utilizing them in strategies. |
| **Recommendations** | Consider acknowledging this issue. |
| **Comments / Resolution** | Acknowledged. |

| Issue_15 | Dust positions can be used to grief strategy-managers |
|---|---|
| **Severity** | **Low** |
| **Description** | Since deposits have no minimum limit, any user can open a position with as few funds as they want. In such a scenario, there will be a very large number of positions that need to be rolled over from epoch to epoch.<br><br>The rollPositions is supplied with an array of positionIds, and loops through them. Since there is no bound on the possible number of positions, the strategy manager can be made to track millions of small positions, and rolling over epochs can become a significant gas cost for the operators. This can be used by malicious users to grief strategies, making them incur severe operational costs. |
| **Recommendations** | Consider adding a minimum deposit limit or acknowledging and documenting this issue. |
| **Comments / Resolution** | Fixed by adding a minimum deposit amount. |

| | |
|---|---|
| Issue_16 | Repayment of borrowed funds via strategyDeposit reverts when the protocol is paused, or the asset is deprecated |
| **Severity** | **Low** |
| **Description** | The strategyDeposit function allows the strategy manager to return underlying assets that were previously borrowed for yield generation via withdrawForLending. However, this function strictly enforces vaultConfig.onlyProtocolAndVaultOn and _requireSupportedAsset(asset). This creates a severe liquidity lock in emergency scenarios, during vault wind-downs, or when an asset is deprecated, as the Strategy Manager is explicitly prevented from repatriating funds. To allow users to retrieve their funds during a pause, the protocol features an emergencyFulfillClaim function that intentionally bypasses the onlyProtocolAndVaultOn check. However, for these emergency claims to succeed, the vault must hold sufficient underlying liquidity.<br><br>If the Strategy Manager holds a significant portion of the vault's assets in external strategies, they must return them using strategyDeposit. Because strategyDeposit calls vaultConfig.onlyProtocolAndVaultOn, the transaction will revert if the protocol is paused. The Strategy Manager becomes entirely incapable of returning the funds, rendering the protocol functionally insolvent and breaking the emergencyFulfillClaim mechanism via InsufficientLiquidity reverts. Similarly, if an admin deprecates an asset by setting it to unsupported, the _requireSupportedAsset(asset) check in strategyDeposit will permanently block the Strategy Manager from returning any outstanding debt of that specific asset. |
| **Recommendations** | Consider removing the vaultConfig.onlyProtocolAndVaultOn and _requireSupportedAsset(asset) checks from the strategyDeposit function, or acknowledging this issue. |
| **Comments / Resolution** | Fixed by removing the checks. Strategy manager is trusted. |

| Issue_17 | Frontrunning deposits to hit maxCapacity can cause targeted DOS |
|---|---|
| **Severity** | **Low** |
| **Description** | When a user submits a deposit transaction, the Vault checks if the incoming funds will push the total TVL over the defined maxCapacity. If an attacker identifies a pending deposit that will bring the vault close to its cap, they can submit a competing transaction with a higher gas price to deposit a minimal amount of assets first.<br><br>When the attacker's transaction is mined, the vault's TVL increases slightly. When the victim's transaction is subsequently processed in the same block, the sum of the new TVL and the victim's deposit will exceed the maxCapacity, triggering a revert. This allows a malicious actor to continuously grief specific users or large depositors. |
| **Recommendations** | Consider acknowledging this issue. |
| **Comments / Resolution** | Acknowledged. |

| | |
|---|---|
| Issue_18 | emergencyWithdrawUnderlying reverts for unsupported assets hindering emergency rescue operations |
| **Severity** | **Low** |
| **Description** | The emergencyWithdrawUnderlying function in Vault.sol is designed to allow authorized roles to extract funds during critical incidents. However, the function strictly enforces the _requireSupportedAsset(asset) modifier. This creates a severe operational blocker: if an asset is deprecated or deemed unsafe, administrators cannot use the emergency withdrawal mechanism to rescue the funds without temporarily re-enabling support for the compromised asset.<br><br>While user-facing emergency functions like emergencyFulfillClaim correctly omit the supported asset check to allow users to exit, emergencyWithdrawUnderlying mandates it. |
| **Recommendations** | Consider removing the _requireSupportedAsset(asset) validation from the emergencyWithdrawUnderlying function. |
| **Comments / Resolution** | Fixed by removing the check. |

| Issue_19 | Emergency requests can exceed the vault asset balance |
|---|---|
| **Severity** | **Informational** |
| **Description** | The Vault.sol contract provides the function emergencyWithdrawUnderlying that allows the emergency redemption requester to create emergency withdrawal requests. However, the function does not check if the total sum of amounts across all pending requests does not exceed the asset balance of the contract. This leads to a revert when reviewEmergencyWithdrawUnderlying is called by the reviewer, as it attempts to withdraw more assets than the contract holds. |
| **Recommendations** | Consider adding logic to limit the cumulative emergency withdrawal request amounts to the contract balance, or acknowledging this issue. |
| **Comments / Resolution** | Acknowledged. |

| Issue_20 | syncStoreCore call can be denied if a deposit is made first |
|---|---|
| **Severity** | **Informational** |
| **Description** | The syncStoreCore function is called to initialize the state of the contracts, setting all the IDs to 1. There is a check to ensure this is done only once.<br><br>*if (store.nextPositionId() == 0) {*<br>*store.setCounters(1, 1, 1);*<br>However, if a user deposits to the vault before this is called, the nextPositionId will be set to 1, and this sync can then never be called again, breaking the assumptions set up by the protocol. |
| **Recommendations** | Ensure the protocol is paused when calling syncStoreCore. |
| **Comments / Resolution** | Acknowledged. |

| Issue_21 | Overpaid funds are silently lost |
|---|---|
| **Severity** | **Informational** |
| **Description** | Strategy managers can repay funds via the strategyDeposit function. If they pay back more than their borrowed amount for an asset, the repaidAmount is truncated.<br><br>*uint256 repaidAmount = received < borrowedByAsset ? received : borrowedByAsset;*<br><br>Thus, in case of over-payments, the code silently consumes the funds and does not update the borrowed state. |
| **Recommendations** | Consider acknowledging and documenting this issue. |
| **Comments / Resolution** | Acknowledged. |

| Issue_22 | Unused notExceedMaxCapacity modifier |
|---|---|
| **Severity** | **Informational** |
| **Description** | The Vault contract implements a notExceedMaxCapacity modifier but never uses it. It is dead code. |
| **Recommendations** | Remove the dead code. |
| **Comments / Resolution** | Fixed by removing the dead code. |

## VaultCommonBase

The VaultCommonBase contract is an abstract helper layer shared by vault implementations. It centralizes common access to VaultStore, VaultConfig, EpochManager, and VaultActivityLogger, and provides reusable helpers for decimal normalization, asset to DRT conversion, position loading and saving, epoch lookups, yield accrual, and bad debt adjusted principal calculations.

Its most important responsibility is handling automatic epoch rollover for active positions. When a position has auto roll enabled, and its epoch has ended, the contract accrues the finished epoch yield, applies any settled bad debt haircut to principal, and then either moves the position into the next open epoch that starts exactly at the previous epoch end or disables auto roll if no valid next epoch exists. The base contract leaves DRT syncing, TVL loss handling, user position recording, and roll events to child contracts through virtual hooks.

#### Core Invariants:

INV 1: Epoch-dependent helpers must only run when an epoch manager reference exists, and any principal reduction for bad debt must use the epoch bad debt value, capped at the basis point denominator.

INV 2: If settled epoch data is required for a calculation, the call must revert unless that epoch has already been settled.

INV 3: Auto roll can only affect active positions with non-zero shares after the current epoch has ended. If there is no next epoch starting at that end time or the next epoch is not open, the position must remain in place, and auto roll must be turned off instead of silently rolling elsewhere.

INV 4: A successful roll must preserve the position owner and move the position into the next epoch only after the current epoch yield has been accrued, bad debt has been applied, and the child contract hooks have handled any DRT, TVL, and logging side effects.

| | |
|---|---|
| Issue_23 | _rollPositionIfNeeded lacks a loop, causing bad debt evasion, lost yield, and locked auto-roll cancellations |
| **Severity** | **Medium** |
| **Description** | The _rollPositionIfNeeded internal function processes auto-rolls by advancing a position exactly one epoch forward. If a position has been inactive for multiple epochs, this single-step logic causes the position's state to stall in an intermediate epoch. This desynchronization allows users to evade bad debt, lose out on yield, and experience permanent reverts when attempting to cancel their auto-roll status.<br><br>When _rollPositionIfNeeded executes, it evaluates p.epochId and looks up the immediate successor epoch via getEpochIdByStartAt(endAt). It applies the yield and bad debt for the current epoch, transitions the position to the single successor epoch, and returns. It does not loop to catch the position up to the current block timestamp.<br><br>If a user's position is multiple epochs behind, interactions with the vault yield broken results:<br><br>1. cancelAutoRoll: The function rolls the position forward by one epoch, then checks if the current block.timestamp is past that intermediate epoch's interestEndAt. Because the intermediate epoch is already deep in the past, this check fails and reverts with CancellationWindowClosed, locking the user's auto-roll state.<br><br>2. fulfillClaim: The position rolls forward by one epoch. Because that intermediate epoch has also already ended, the helper disables autoRoll. The main function then finalizes the withdrawal based entirely on the intermediate epoch. Any subsequent epochs (including their generated yield or realized bad debt) are entirely skipped, allowing users to strategically evade losses. |
| **Recommendations** | Consider refactoring _rollPositionIfNeeded to utilize a while loop that continuously rolls the position forward as long as the current epoch has ended, the position has autoRoll enabled, and a valid |

| | |
|---|---|
| | successor epoch exists. Alternatively, ensure that rollPositions is called at regular intervals to prevent positions from falling multiple epochs behind, and acknowledge this issue. |
| **Comments / Resolution** | Acknowledged. The strategy manager is responsible to make sure operations are done properly. |

| | |
|---|---|
| Issue_24 | Phantom yield can be created due to the yield calculation before the application of debt to the principal |
| **Severity** | **Informational** |
| **Description** | When rolling positions over to a new epoch, the yield percent is applied first, followed by the application of bad debt on the principal. This order of operations is correct; however, it requires the team to configure a correct epochYieldBps to avoid creating non-existent yield.<br><br>For example, if epochYieldBps = 10% for that epoch, principal = 100, hence yield = 10. Now, if bad debt = 10%, principal is cut to 90, so the actual yield should be 9. Due to this, the yield can eat into the principal/yield of other users. |
| **Recommendations** | Consider acknowledging this risk and ensuring the epochYieldBps will be configured correctly. |
| **Comments / Resolution** | Acknowledged. |

## RedeemVault

The RedeemVault contract is the payout and unwind layer for vault positions. It uses VaultStore to track positions and early withdrawal claims, reads epoch data from EpochManager, and handles both standard maturity redemption and reviewed early exits. When a position matures, the contract can fulfill the claim by applying any settled bad debt haircut to principal, burning the holder's DRT for the redeemed principal, crediting the epoch yield into the position, and paying out the matured principal. Yield is claimed separately and may be charged a protocol fee.

For early exits, the contract routes requests through EarlyWithdrawReview before any claim is opened. Once approved, the position is marked as early withdrawn and split into immediate principal, retained principal, seller yield after penalty, and forfeited yield. The seller can claim the immediate portion after a delay and the retained portion plus reduced yield at maturity, while a strategy manager can assign a buyer who later claims the purchased principal exposure together with the forfeited yield. The contract also handles position auto roll transitions, liquidity pulls from the main vault when needed, and TVL updates as principal leaves the system.

### Privileged Functions

- syncStoreCore
- bindVault
- setEarlyWithdrawReview
- ensureNoPendingEarlyWithdraw
- transferReviewFundOut
- onEarlyWithdrawApproved
- onEarlyWithdrawRejected
- takeEarlyWithdrawClaim
- emergencyFulfillClaim

### Core Invariants:

INV 1: Only the vault owner can sync core references, bind a new vault, or set the review contract. Only the review contract can approve or reject early withdrawal requests or move review funds; only the strategy manager can assign a buyer to an approved early withdrawal claim, and only an emergency redeem reviewer can force emergency fulfillment.

INV 2: A position cannot open a new early withdrawal request if it is not active, if its lockup has already ended, or if another request for the same position is still pending. Any seller or buyer payout must reference the approved claim recorded for that exact position.

INV 3: Principal can only leave through the defined redemption paths. Whenever maturity principal, immediate principal, or retained principal is paid out, the corresponding DRT amount is burned, and TVL is reduced by the redeemed principal amount.

INV 4: The early withdrawal split must remain consistent. Immediate principal plus retained principal must equal the original principal, and the forfeited yield pool is partitioned across seller payout reduction, buyer reward, and protocol fee without double-counting. Each claim leg can only be collected once and only after its required delay or maturity.

| | |
|---|---|
| Issue_25 | Withdraw requests approved just before maturity can lead to stuck positions |
| **Severity** | **High** |
| **Description** | When an early withdrawal is required, the user calls earlyWithdraw. This creates a request which then needs to be deposited against and then reviewed by someone with an appropriate role. Once accepted via the reviewEarlyWithdrawRequest function, the strategy manager can assign a buyer to that position, which then completes the process.<br><br>The issue is that since this process can take over two weeks (reviewEarlyWithdrawRequest), it can happen that the reviewer approves the request, and the epoch hits maturity, before the strategy manager has the opportunity to assign a buyer.<br><br>In this scenario, the position gets soft-locked. Due to the following check, the strategy manager is unable to assign a buyer in the takeEarlyWithdrawClaim function.<br><br>*if (block.timestamp >= t.maturity) revert Errors.CooldownEnded();*<br><br>The strategy manager is supposed to deposit the withdrawn funds via this function, which then compensates the reviewer depositor, but that is not possible anymore. Furthermore, since no buyer can be assigned, the forfeited yield is now lost since no one can claim it.<br><br>This leads to a stuck position and unclaimable funds in the system, and there are no code restrictions preventing this scenario. |
| **Recommendations** | Consider capping the reviewDeadline to epochEndAt to ensure it never exceeds the epoch's maturity. And if maturity is hit for a position with no buyer, the forfeited yield should be claimable by someone. |
| **Comments / Resolution** | Fixed following recommendations. |

| Issue_26 | Review funds can get stuck in the system |
|---|---|
| **Severity** | **High** |
| **Description** | In the redeem contract, before any token transfer, the _ensurePayoutLiquidity function is called to make sure the contract is liquid. However, this function is not called in the transferReviewFundOut function.<br><br>Due to this, withdrawEarlyWithdrawReviewFund calls can fail. This is because when reviewers deposit funds, the tokens are sent to the redemption contract. However, these funds are not kept separate and can be used up for redemption processing and yield claims. So when the withdrawEarlyWithdrawReviewFund function is called, the redeem contract can actually be empty, holding no funds.<br><br>This leads to transaction failures and stuck funds, since the reviewer's funds cannot be recovered anymore. |
| **Recommendations** | Add a _ensurePayoutLiquidity call in the transferReviewFundOut function. |
| **Comments / Resolution** | Fixed following recommendations. |

| Issue_27 | Bad debt can be charged twice |
|---|---|
| **Severity** | **High** |
| **Description** | In the _fulfillMatureClaimByPosition function, the position is autorolled.<br><br>*p = _rollPositionIfNeeded(positionId, p);*<br>*if (p.autoRoll) revert Errors.PositionAutoRolled();*<br><br>If no suitable epoch is found to roll into, the autoRoll value is set to false, passing the check shown above.<br><br>*if (newEpochId == 0 ||*<br>*!_epochManagerRef().isEpochOpen(newEpochId)) {*<br>*p.autoRoll = false;*<br><br>In such a scenario, the position's principal has already been reduced by the bad debt APY.<br><br>The function, however, assumes that the position hasn't been rolled over at all, and thus assumes no bad debt is applied, and applies the bad-debt again.<br><br>*uint256 principalAfterBadDebt =*<br>*_principalAfterEpochBadDebt(p.epochId, principalPart, true);*<br><br>So in this scenario, the bad debt gets applied twice, leading to a double loss of the user's position. |
| **Recommendations** | Bad debt should only be charged once. Consider updating _rollPositionIfNeeded in VaultCommonBase to check for a valid open successor epoch before applying any bad debt or yield calculations. |
| **Comments / Resolution** | Fixed following recommendations. |

| | |
|---|---|
| Issue_28 | Retained principal is not treated as first-loss on the seller's claim causing Vault overpayment during bad-debt epochs |
| **Severity** | **High** |
| **Description** | When a position is early-withdrawn, it is split into immediatePrincipal (e.g. 90 %, taken over by a buyer) and retainedPrincipal (e.g. 10 %, held back as the seller's first-loss buffer). The post-audit design intends that if bad debt hits the epoch, the seller's retainedPrincipal absorbs the loss first, and only the overflow beyond retainedPrincipal is passed on to the buyer.<br><br>The buyer-side payout in claimEarlyWithdrawBuyer was updated to implement this waterfall correctly, computing buyerBadDebt = max(0, totalBadDebt – retainedPrincipal). However, the seller-side payout in _claimEarlyWithdrawSellerByClaim was not updated and still applies a proportional bad-debt cut to retainedPrincipal as retainedPrincipal × (1 – badDebtBps/BPS).<br><br>Because of this inconsistency, both sides partially absolve the bad debt that falls on the retained portion: the buyer is relieved because the seller's retained principal is presumed to absorb it, while the seller is simultaneously paid back most of that retained principal as if the waterfall did not apply. For a 1000 USDC position with a 90/10 split and 10 % bad debt, the buyer receives the full 900 and the seller receives 90, so the protocol pays out 990 for a position whose actual post-bad-debt value is 900. The excess 90 is pulled from Vault cash via _ensurePayoutLiquidity, silently diluting maturity holders in the same epoch. |
| **Recommendations** | Make sure retainedPrincipal truly acts as the first-loss layer.<br><br>Note that once the fix is applied, the seller's payout may be 0 when totalBadDebt ≥ retainedPrincipal, removing their incentive to call the retained claim. Because this function also burns the remaining DRT, decrements TVL, and finalizes the position state, an abandoned claim leaves the position as a zombie EarlyWithdrawApproved forever, with dangling DRT and an overstated TVL that silently shrinks the vault's effective |

| | |
|---|---|
| | maxCapacity. Consider adding a functionality so the state cleanup can proceed regardless of whether the seller shows up. |
| **Comments / Resolution** | Fixed following recommendations. However if the bad debt causes. |

| | |
|---|---|
| Issue_29 | Early withdrawers can avoid bad debt |
| **Severity** | **Medium** |
| **Description** | Early withdrawers can avoid paying bad debt in an epoch since they get a portion of their funds back guaranteed. Only 10% of their principal is held, and a penalty is applied on that amount. The remaining 90% is claimable immediately and can be taken out without any losses.<br><br>The losses are instead borne by the buyers of the position. The buyer pays the immediatePrincipal amount, but can only claim _principalAfterEpochBadDebt and yield.<br><br>Thus, quick users can avoid debt socialization by exiting early and placing the burden on the buyers. |
| **Recommendations** | Consider acknowledging this issue, since the buyer must be cautious of buying debts in bad-debt epochs. |
| **Comments / Resolution** | Partially fixed.Although buyer side payout may have been updated to account for retainedPrincipal as a first loss buffer, the seller side retained claim still appears to pay retained principal using only a proportional bad debt haircut.<br><br>This creates a separate overpayment issue, see issue;<br><br>*Retained principal is not treated as first-loss on the seller's claim causing Vault overpayment during bad-debt epochs* |

| Issue_30 | The borrowed amount can be off due to early withdrawals |
|---|---|
| **Severity** | **Medium** |
| **Description** | During normal operations, the strategy manager borrows the deposited tokens and uses them to generate yield. The same tokens are then repaid to the vault, which are then used to process the withdrawals. The key point is that when funds are transferred from the strategy manager to the vault for withdrawals, the borrowed amount is decremented.<br><br>*store.setBorrowedAmountByAsset(asset, borrowedByAsset - repaidAmount);*<br>*store.setBorrowedAmount(borrowedAmount - normalizedRepaid);*<br><br>However, this is not true for early withdrawals. In early withdrawals, the reviewer fronts funds to the user. Then, via the takeEarlyWithdrawClaim function, the strategy manager sends funds to the redeem vault, repaying the reviewer. However, in this flow, even though the strategy manager sends funds to process withdrawals, the borrowed amount is never updated. Thus, the borrowed amount will always be inflated. |
| **Recommendations** | Consider adjusting the borrow amounts in the early withdrawal flow as well. Furthermore, consider utilizing the free funds present in the system from deposits to process early withdrawals before requiring funds from the strategy manager. |
| **Comments / Resolution** | Fixed takeEarlyWithdrawClaim now sends funds to the vault instead of redeem vault.. |

| Issue_31 | Unburnt DRT tokens in the early withdrawal process |
|---|---|
| **Severity** | **Medium** |
| **Description** | In normal withdrawals via _fulfillMatureClaimByPosition, the entire DRT amount is burnt, irrespective of bad debt adjustments.<br><br>*uint256 sharesToBurn = p.shares;*<br>*IVaultDRTRef(address(_drtTokenRefByEpoch[p.epochId])).burn(p.owner, sharesToBurn);*<br><br>This ensures that no DRT tokens related to that position remain. However, in the early withdrawal flow, these DRT tokens are burnt in the _claimEarlyWithdrawImmediate and _claimEarlyWithdrawSellerByClaim functions, but on the actual payment amount, which can be lower than the initial mint in case that position has been rolled through epochs with bad debt.<br><br>*_burnDrtForPrincipal(t.seller, t.epochId, ua, t.retainedPrincipal);*<br>*_burnDrtForPrincipal(t.seller, t.epochId, ua, t.immediatePrincipal);*<br><br>They should instead burn the original number of shares minted to ensure that no DRT tokens remain.<br><br>As an example, say Alice deposited 100 USDC tokens and was minted 100 DRT tokens. Then, in an epoch with 10% bad debt, Alice's principal reduces to 90, but she still has 100 DRT tokens. Then, in early withdrawal, only 90 DRT tokens are burnt, since that is her principal now. 10 DRT tokens still stay with Alice, unburnt. |
| **Recommendations** | Ensure all DRT tokens are burnt when the position is closed. This can be done by reducing the p.shares amount in the immediate claim, and then burning off all p.shares tokens in the seller claim. |
| **Comments / Resolution** | Fixed following recommendations. |

| | |
|---|---|
| Issue_32 | Static yield calculation during early withdrawal causes insolvency if the settled APY diverges from the initial APY |
| **Severity** | **Medium** |
| **Description** | When an early withdrawal request is processed, the contract calculates and permanently locks the exact amount of accrued and forfeited yield based on the epoch's initial expected APY. However, because the protocol allows the true APY to be adjusted later via settleEpoch, locking in static yield amounts during early exits fundamentally breaks the vault's solvency if the strategy's actual returns differ from the projected returns.<br><br>During earlyWithdraw, the RedeemVault fetches the current epochApyBps and uses it in _calcEarlyWithdrawTerms to determine absolute integer values for sellerYieldAfterPenalty, forfeitedYieldGross, and the protocol fee. Once the reviewer approves the request, these static integers are permanently saved in the EarlyWithdrawClaim struct.<br><br>At the end of the epoch, the EpochManager allows the strategy manager to call settleEpoch, which updates the epoch's final apyBps to reflect the actual yield generated by the external strategy. When the early seller and the buyer later claim their respective portions at maturity, the contract unconditionally pays out the statically saved yield variables.<br><br>If the strategy underperforms and the settled APY is significantly lower than the initial APY, the contract will pay the early seller and the buyer the yield that does not actually exist in the vault. This overpayment will drain the legitimate yield belonging to users who held their positions to maturity, resulting in InsufficientLiquidity reverts when those standard users attempt to execute fulfillClaim. |
| **Recommendations** | Consider calculating the yield using the apyBps only after the epoch has been settled for early withdrawals as well, and split it into yield for the seller and buyer after the epoch has been settled. Or ensure that apyBps cannot be revised downwards during epoch settlement. |

| | |
|---|---|
| **Comments / Resolution** | Partially fixed. The fix scales early withdrawal yield by comparing settled APY against earlyWithdrawClaimInitialApyBps. The problem is that this “initial” APY is not stored when the early withdrawal terms are calculated. It is stored later during approval by re-reading the epoch APY. Because approval is still allowed when block.timestamp == reviewDeadline, and settlement is also allowed when block.timestamp == epochEndAt, the epoch can be settled first at the exact boundary. Approval then records the already lowered settled APY as the “initial” APY, so the scaling check sees no drop and pays the old locked yield.<br><br>Possible resolution:<br><br>Store the APY used to calculate the early withdrawal terms at request creation time. Pass that value through the review request and use it on approval instead of reading the epoch APY again. |
| **Comments / Resolution 2** | Fixed. |

| | |
|---|---|
| Issue_33 | Funds can get stuck/orphaned in the redeemVault in early withdrawal flows |
| **Severity** | **Medium** |
| **Description** | Funds can move from the vault to the redeemVault, but there is no function to move funds from the redeemVault back to the Vault. Thus, any unaccounted-for funds in the redeemVault are stuck there, even if the Vault contract expects access to these funds.<br><br>This same situation can arise in early withdrawal flows involving bad debt. Say an early withdrawal is ordered for 100 USDC (in a 1000 USDC vault). The reviewer pays 100 USDC to the redeemVault, which is then collected by the user via _claimEarlyWithdrawImmediate. Then the strategy manager sends 100 USDC tokens to the redeemVault via takeEarlyWithdrawClaim. Later, when the buyer tries to claim the tokens, however, they are only given the amount after absorbing bad debt (say 10%).<br><br>*uint256 immediateAfterBadDebt = _principalAfterEpochBadDebt(t.epochId, t.immediatePrincipal, true);*<br>*uint256 amount = immediateAfterBadDebt + t.forfeitedYield;*<br>*ua.safeTransfer(msg.sender, amount);*<br><br>So, for a bad debt of 10%, the buyer only receives 90 tokens. This means 10 tokens, which the strategy manager had already sent to the redeemVault, are now stuck there. These tokens cannot be recovered and will slowly flow out via normal redemptions and yield claims.<br><br>After the haircut, the vault/strategy manager expects to hold 810 tokens (900 due to bad debt, -90 due to the withdrawal). However, they only hold 800 tokens, and 10 tokens are in the redeemVault, which cannot be borrowed to generate yield. |
| **Recommendations** | Allow the strategy manager to send funds from the redeemVault to the vault. This should be fine since all transfer functions in the redeemVault call _ensurePayoutLiquidity before paying out. Note: the transferReviewFundOut function does not call |

| | |
|---|---|
| | _ensurePayoutLiquidity, which is an issue in itself and must also be fixed if this fix is being applied. |
| **Comments / Resolution** | Fixed following recommendations. |

| | |
|---|---|
| Issue_34 | Selective epoch rolling via enableAutoRoll allows retroactive bad-debt avoidance |
| **Severity** | **Medium** |
| **Description** | The enableAutoRoll function allows a user to re-enable auto-rolling on a position whose epoch ended arbitrarily long ago. Since all intermediate epochs are already settled with on-chain bad-debt and APY data, the user can inspect outcomes before deciding to roll. By repeatedly calling enableAutoRoll followed by cancelAutoRoll (which triggers _rollPositionIfNeeded internally), the user advances one epoch per cycle, collecting yield at each step. They can stop before any epoch with unfavorable bad debt and call fulfillClaim to exit, effectively cherry-picking which epochs to participate in with full hindsight.<br><br>For example, a position stuck at epoch 5 when the current epoch is 10 can roll through epochs 6 and 7 (0% bad debt, collecting yield) and stop before epoch 8 (20% bad debt). The user's share of epoch 8's bad debt goes unabsorbed and falls on the Vault's general cash pool, creating a shortfall for maturity holders.<br><br>This is possible because enableAutoRoll has no check that the position's epoch is currently active, and because cancelAutoRoll has an early-return path that bypasses its interestEndAt window check when the roll itself sets autoRoll to false. |
| **Recommendations** | Rather than exposing a general-purpose enableAutoRoll, automatically restore autoRoll when a rejected or expired early-withdraw request is detected. Or consider acknowledging the initial issue and document that users need to open a new position to re-enable autoRoll. |
| **Comments / Resolution** | Fixed be removing enableAutoroll functionality. |

| Issue_35 | takeEarlyWithdrawClaim sends funds to the vault, which can be borrowed out immediately |
|---|---|
| **Severity** | **Low** |
| **Description** | The takeEarlyWithdrawClaim function has been modified to send the funds to the vault instead of the RedeemVault. This is called by the strategy manager to pay off the early withdrawal claim buyer. However, the funds are sent to the vault instead of being kept in the RedeemVault.<br><br>The issue with this is that these funds can then immediately be borrowed by the strategy manager by calling withdrawForLending. withdrawForLending allows the full balance of the vault to be loaned out, including these funds.<br><br>The same issue also applies to the sweepToVault function, which can move funds required for payments into the vault and can be immediately loaned out. There are no further constraints in the withdrawForLending function, so any tokens present can be taken out. |
| **Recommendations** | Consider acknowledging this issue since StrategyManager is trusted and is thus responsible for making sure enough funds stay in the vault to make pending payments. Otherwise, set up an accounting system that keeps track of the actual free funds amount in the vault contract and only allows borrows up to that limit. |
| **Comments / Resolution** | Acknowledged. |

| Issue_36 | autoRoll is permanently disabled for a position if an early withdrawal request is rejected |
|---|---|
| **Severity** | **Low** |
| **Description** | When a user requests an early withdrawal, the protocol proactively disables their position's autoRoll flag to prevent it from rolling into a new epoch while under review. However, if the reviewer rejects the request, the protocol fails to restore the autoRoll state. Because there is no function to re-enable autoRoll, the user permanently loses the auto-rollover functionality for that position. |
| **Recommendations** | Consider implementing functionality that allows a user to re-enable autoRoll if their early withdrawal request has been rejected. |
| **Comments / Resolution** | Fixed, but the fix appears to introduce a new issue.<br><br>A general purpose enableAutoRoll can allow users to retroactively roll stale positions through already settled epochs and avoid unfavorable bad debt epochs.<br><br>See new finding:<br>*Selective epoch rolling via enableAutoRoll allows retroactive bad-debt avoidance*<br><br>enableAutoroll function has been removed. |

| Issue_37 | Strategy manager can set any buyer |
|---|---|
| **Severity** | **Low** |
| **Description** | In the early withdrawal flow, the reviewer deposits their funds to facilitate the early exit. Later, the strategy manager funds the contract via takeEarlyWithdrawClaim, and specifies a buyer who can then claim the principal amount + yield.<br><br>The issue is that the strategy manager can set any buyer, and there is nothing ensuring that the buyer is the original depositor. Thus, if the strategy manager sets it to some other address, the reviewer will pay for the exit, but some other buyer gets the rewards. |
| **Recommendations** | Consider setting the buyer as the original depositor for the exit, or acknowledge this issue. |
| **Comments / Resolution** | Acknowledged. |

| | |
|---|---|
| Issue_38 | Changing earlyWithdrawReview address locks positions with pending requests |
| **Severity** | **Informational** |
| **Description** | When a user requests an early withdrawal, the request data is stored in the active EarlyWithdrawReview contract, while the RedeemVault stores the requestId against the user's position in pendingEarlyWithdrawRequestIdByPosition. If the admin updates the earlyWithdrawReview address via setEarlyWithdrawReview, the RedeemVault begins pointing to a fresh contract with an empty state.<br><br>If a user with a pending request attempts to interact with their position (e.g., via fulfillClaim, earlyWithdraw, or cancelAutoRoll), the vault calls _ensureNoPendingEarlyWithdraw. This function queries the new review contract for the request's status. Because the new contract lacks the historical data, it reverts with Errors.VaultRecordNotFound. This unhandled revert completely bricks the user's position. |
| **Recommendations** | Consider acknowledging this issue or making sure that the earlyWithdrawReview address cannot be changed while there are pending requests. |
| **Comments / Resolution** | Acknowledged. |

| Issue_39 | Position state is not updated to Redeemed, and balances are not zeroed after early withdrawal completes |
|---|---|
| **Severity** | **Informational** |
| **Description** | After a user completes an early withdrawal by claiming both their immediate and retained principal, their underlying Position struct is never updated to reflect the exit. The position retains its original principalAmount, its original shares, and remains indefinitely stuck in the EarlyWithdrawApproved state.<br><br>In a standard maturity withdrawal (_fulfillMatureClaimByPosition), the contract correctly cleans up the user's state by setting p.shares = 0, p.principalAmount = 0, and p.status = VaultStore.PositionStatus.Redeemed. |
| **Recommendations** | Consider updating the position state after the full early withdrawal completes. |
| **Comments / Resolution** | Fixed. |

| Issue_40 | Redundant fee > 0 check in _transferProtocolFee |
|---|---|
| **Severity** | **Informational** |
| **Description** | The _transferProtocolFee function begins with an early return pattern:<br><br>*if (fee == 0) return;*<br><br>This guarantees that if execution proceeds past the first line, the fee variable is strictly greater than zero. However, before executing the transfer, the function performs the exact same logical evaluation again via an if (fee > 0) block wrapper. |
| **Recommendations** | Remove the redundant if (fee > 0) wrapper |
| **Comments / Resolution** | Fixed following recommendations. |

| Issue_41 | Emit event early in bindVault |
|---|---|
| **Severity** | **Informational** |
| **Description** | The VaultBound event can be emitted early to avoid the creation of a memory variable to save gas.<br><br>*function bindVault(address newVault) external {*<br>*vaultConfig.onlyVaultOwner(msg.sender);*<br>*_notZeroAddress(newVault);*<br>*address old = vault;*<br>*vault = newVault;*<br>*emit VaultBound(old, newVault);*<br>*}* |
| **Recommendations** | Emit the event early and delete the memory variable as follows:<br><br>*function bindVault(address newVault) external {*<br>*vaultConfig.onlyVaultOwner(msg.sender);*<br>*_notZeroAddress(newVault);*<br>*emit VaultBound(vault, newVault);*<br>*vault = newVault;*<br>*}* |
| **Comments / Resolution** | Fixed following recommendations. |

| Issue_42 | Missing activity check for protocol and vault status |
|---|---|
| **Severity** | **Informational** |
| **Description** | Functions claimYield, claimEarlyWithdrawBuyer in RedeemVault do not check if the protocol and vault are on by calling the onlyProtocolAndVaultOn function on the VaultConfig.<br><br>*function onlyProtocolAndVaultOn() external view virtual {*<br>*if (dowConfig().paused()) revert Errors.ProtocolPaused();*<br>*address vaultAddress = store.vault();*<br>*if (vaultAddress == address(0)) revert Errors.VaultClosed();*<br>*if (!IVault(vaultAddress).isVaultOpen()) revert Errors.VaultClosed();*<br>*}* |
| **Recommendations** | Consider calling the function to disallow user actions during the protocol-paused state, or acknowledging and documenting the issue. |
| **Comments / Resolution** | Fixed<br><br>However, claimEarlyWithdrawImmediate does not disallow user actions during the protocol-paused. Consider checking onlyProtocolAndVaultOn if unintended behaviour. |

## EarlyWithdrawReview

The EarlyWithdrawReview contract is the review and funding coordinator for early withdrawal requests. It uses AccessControl to separate vault approval from reviewer management, allowing approved vaults to submit requests while only designated reviewers for a given vault can fund, approve, reject, or withdraw review funds.

Its core workflow is that an approved vault creates a pending request, a reviewer deposits the exact immediate principal amount into the vault as review funding, and then that reviewer can approve or reject the request. Approval requires the full review fund to be present and triggers the vault callback that opens the early withdrawal claim. Rejection triggers the vault rejection callback. Pending requests automatically read as rejected after the review deadline passes, and reviewer actions are recorded for later inspection.

### Privileged Functions

- setVaultApproved
- setReviewer
- createRequest
- depositEarlyWithdrawReviewFund
- withdrawEarlyWithdrawReviewFund
- reviewEarlyWithdrawRequest

### Core Invariants:

INV 1: Only approved vaults can create requests, and only reviewers explicitly enabled for a given approved vault can fund, withdraw, approve, or reject that vault's requests.

INV 2: A request can only be approved while it is still pending and only if the total review fund for that request exactly matches the immediate principal required by the request.

INV 3: A request id must be unique per vault, and once a request is approved, its status, reviewer, review timestamp, and approved claim id must be recorded together with the review fund settlement.

INV 4: Review funds cannot be withdrawn after approval, but they can be withdrawn when the request is not approved, including when a pending request has effectively expired past its review deadline and is treated as rejected.

| Issue_43 | Duplicate request IDs accepted by createRequest from different vaults |
|---|---|
| **Severity** | **Medium** |
| **Description** | The function createRequest in EarlyWithdrawReview accepts calls only from approved vaults. If the call arrives from a future authorized vault that shares the same requestId from another vault, the function will proceed to update userRequestIds and allRequestRefs with duplicate requestIds. |
| **Recommendations** | Consider storing userRequestIds in a per-vault mapping. |
| **Comments / Resolution** | Fixed following recommendations. |

| Issue_44 | Removed reviewers cannot claim withdrawals |
|---|---|
| **Severity** | **Low** |
| **Description** | A reviewer can deposit funds to process early withdrawals via the depositEarlyWithdrawReviewFund function. If the withdrawal is not accepted, they can retrieve their funds via the withdrawEarlyWithdrawReviewFund function.<br><br>However, the withdrawEarlyWithdrawReviewFund function has the _requireReviewerForVault check, which means that only valid reviewers can claim the funds back. Thus, if a reviewer deposits funds and then gets removed as a valid reviewer, they will lose their ability to claim back their deposit. |
| **Recommendations** | Consider allowing anyone to claim back deposits or acknowledging this issue. |
| **Comments / Resolution** | Fixed following recommendations. |

| Issue_45 | Stale vault reviewers if the vault is removed |
|---|---|
| **Severity** | **Informational** |
| **Description** | If a vault is unapproved by the VAULT_MANAGER_ROLE, the reviewers of the vault (stored in the reviewersByVault mapping) are not cleared and remain stale.<br><br>*function setVaultApproved(*<br>*address vault,*<br>*bool approved*<br>*) external onlyRole(VAULT_MANAGER_ROLE) {*<br>*if (vault == address(0)) revert Errors.InvalidZeroAddress();*<br>*approvedVaults[vault] = approved;*<br>*emit VaultApprovalUpdated(vault, approved);*<br>*}* |
| **Recommendations** | Consider acknowledging this issue. |
| **Comments / Resolution** | Acknowledged. |